WEEK 1

Software Developer Interview Question Bank

132 common practice questions for entry-level developer interviews in India

Built for your situation
Backend / full-stack direction • MERN-oriented preparation • 30 minutes per day

How to use this guide

- Attempt each question before checking notes or searching for an explanation.
- Practise explaining your answer aloud in your own words.
- For coding questions, write and test code rather than memorising a solution.
- Mark questions Green (independent), Yellow (partly understood), or Red (not understood).
- This is a representative practice bank, not a guarantee that every question will be asked by a particular employer.

Important: 132 questions is the full bank, not a demand to master all 132 in one sitting. Use the 10-day plan near the end to pace your preparation.

1. Introduction and HR questions

1. Tell me about yourself.
2. Walk me through your resume.
3. Why did you choose Computer Science?
4. Why do you want to become a Software Developer?
5. What are your strongest technical skills?
6. What are your biggest strengths and weaknesses?
7. Why should we hire you as a fresher?
8. What makes you different from other candidates?
9. What have you been learning recently?
10. Tell me about a technical challenge you faced and how you handled it.
11. How do you approach a problem when you don't know the solution?
12. Where do you see yourself in three years?
13. Why do you want to join our company?
14. Are you comfortable learning unfamiliar technologies?
15. Do you have any questions for the interviewer?

2. Resume and project questions

16. Explain your most important project in two minutes.
17. What problem does your project solve?
18. Why did you choose this particular technology stack?
19. Explain the architecture of your project.
20. Walk me through what happens when a user makes a request to your backend.
21. How is your database designed?
22. Which APIs have you implemented, and what does each one do?
23. How did you implement authentication and authorization?
24. What was the most difficult bug you encountered?
25. How did you debug that issue?
26. What technical decision would you change if you rebuilt the project?
27. How did you test your application?
28. How is your project deployed?
29. What security risks does your application have?
30. What would you improve if your application had 10,000 users?
31. Which parts of the project did you personally implement?
32. Can you explain one important function in your project line by line?

3. Programming fundamentals

33. What is the difference between a compiler and an interpreter?

34. What is the difference between a variable and a constant?
35. What are the main principles of object-oriented programming?
36. What is the difference between a class and an object?
37. What is inheritance, and when would you use it?
38. What is polymorphism?
39. What is encapsulation?
40. What is abstraction?
41. What is the difference between an array and a linked list?
42. What is the difference between a stack and a queue?
43. What is recursion?
44. What is the difference between pass-by-value and pass-by-reference?
45. What is exception handling?
46. What is the difference between syntax errors, runtime errors, and logical errors?

4. JavaScript fundamentals

47. What is the difference between var, let, and const?
48. What is the difference between == and ===?
49. What are undefined, null, and NaN?
50. What is hoisting in JavaScript?
51. What is a closure?
52. What is the difference between a normal function and an arrow function?
53. How does the this keyword work?
54. What is the difference between synchronous and asynchronous execution?
55. What are callbacks, promises, and async/await?
56. What is the JavaScript event loop?
57. What is the difference between map(), filter(), and reduce()?
58. What is destructuring?
59. What is the difference between spread syntax and rest parameters?
60. What is the difference between shallow copying and deep copying?
61. What happens when a promise is rejected?
62. What is the difference between local scope, function scope, and global scope?

5. DSA and problem-solving

63. What is a data structure, and why do we need one?
64. What is an algorithm?
65. What is time complexity?
66. What is space complexity?
67. What does Big O notation represent?
68. What is the difference between O(n) and O(log n)?

69. What is the difference between linear search and binary search?
70. How does sorting help with problem-solving?
71. When would you use a hash map?
72. How would you find the largest and second-largest elements in an array?
73. How would you reverse a string without using a built-in reverse method?
74. How would you determine whether a string is a palindrome?
75. How would you find duplicate elements in an array?
76. How would you count the frequency of each element in an array?
77. How would you check whether two strings are anagrams?
78. How would you find a missing number in a sequence?
79. How would you remove duplicates from an array?
80. How would you explain your solution's time and space complexity?

6. Backend development

81. What happens when you enter a URL into a browser?
82. What is the client-server architecture?
83. What is an API?
84. What is a REST API?
85. What is the difference between GET, POST, PUT, PATCH, and DELETE?
86. What are HTTP status codes, and when would you use 200, 201, 400, 401, 403, 404, and 500?
87. What is middleware in Express.js?
88. How does routing work in Express?
89. What is the difference between authentication and authorization?
90. What is JWT, and how does it work?
91. Why should passwords be hashed instead of stored as plain text?
92. What is CORS?
93. How should a backend handle errors?
94. What is input validation, and why is it necessary?
95. What is the difference between frontend validation and backend validation?
96. What are environment variables, and why should secrets not be hardcoded?
97. What is the difference between synchronous and asynchronous operations in Node.js?
98. How would you debug an API that returns an unexpected error?

7. DBMS and SQL

99. What is a database, and what is a DBMS?
100. What is the difference between SQL and NoSQL?
101. What is a primary key?
102. What is a foreign key?
103. What is the difference between WHERE and HAVING?

104. What is the difference between INNER JOIN and LEFT JOIN?
105. What is database normalization?
106. What is an index, and what are its trade-offs?
107. What are ACID properties?
108. What is the difference between DELETE, TRUNCATE, and DROP?
109. How would you find duplicate records using SQL?
110. How would you find the second-highest salary in a table?
111. How would you retrieve the top three highest-paid employees?
112. When would you choose MongoDB over a relational database?
113. How would you model users and orders in a database?

8. Core computer science fundamentals

114. What is an operating system?
115. What is the difference between a process and a thread?
116. What is deadlock?
117. What is virtual memory?
118. What is the difference between TCP and UDP?
119. What is DNS?
120. What is the difference between HTTP and HTTPS?
121. What happens during a TCP connection handshake?
122. What is the difference between RAM and persistent storage?
123. What is concurrency, and how is it different from parallelism?

9. Behavioural and situational questions

124. Tell me about a time you struggled to learn something technical.
125. Describe a mistake you made and what you learned from it.
126. Tell me about a time you had to finish work under a deadline.
127. What would you do if a teammate disagreed with your technical decision?
128. What would you do if you could not solve a production bug?
129. How do you decide what to learn next?
130. How do you respond to critical feedback?
131. What would you do if you were assigned a task using a technology you had never used?
132. How do you balance learning theory with building projects?

10-Day Plan for 30 Minutes a Day

The aim is to review every question once, go deeper on the highest-value topics, and finish with recall practice. Do not rush just to tick a box.

Day	Questions	Focus	30-minute target
Day 1	Questions 1–15	Introduction and HR	Choose 2–3 answers to practise aloud; scan the rest and mark weak ones.
Day 2	Questions 16–32	Resume and projects	Prepare your strongest project story and understand every resume claim.
Day 3	Questions 33–46	Programming fundamentals	Explain core concepts simply and connect each to a small example.
Day 4	Questions 47–62	JavaScript fundamentals	Prioritise var/let/const, equality, closures, this, async, promises, and scope.
Day 5	Questions 63–80	DSA and problem-solving	Attempt one problem in code; briefly review the remaining question prompts.
Day 6	Questions 81–98	Backend development	Prioritise REST, HTTP methods/status codes, middleware, JWT, validation, and debugging.
Day 7	Questions 99–113	DBMS and SQL	Understand keys, joins, normalization, indexes, ACID, and attempt one SQL query.
Day 8	Questions 114–123	Core CS fundamentals	Focus on clear, concise explanations of OS and networking basics.
Day 9	Questions 124–132	Behavioural questions	Build honest examples from study, projects, teamwork, deadlines, and learning.
Day 10	Review all marked questions	Recall and mini mock	Answer without notes; retry Red/Yellow questions and deliver a 2-minute project explanation.

A realistic 30-minute routine

- 5 minutes: recall yesterday's Yellow and Red questions without notes.
- 15 minutes: work through the day's question range. For unfamiliar prompts, identify what you need to learn and record one concise note.
- 7 minutes: go deeper on one high-priority answer or one small coding example.
- 3 minutes: mark Green / Yellow / Red and write the next action.

Do not try to write 15 polished answers in 30 minutes. On days with many prompts, review the full range quickly, then deeply practise only the most important questions. If your understanding is weak, carry questions forward instead of pretending they are completed.

Suggested first priorities

- Introduction and resume: tell me about yourself; walk me through your resume; explain your strongest project.
- JavaScript: var/let/const; == versus ===; closures; synchronous versus asynchronous execution; promises and async/await.

- Problem-solving: largest / second-largest element; reverse a string; palindrome; duplicates; frequency counting.
- Backend and data: REST APIs; HTTP methods and status codes; Express middleware; authentication versus authorization; SQL joins and keys.

Interview readiness is demonstrated by what you can explain, implement, and defend—not by how many questions you have read.